



COS6012-B: Concurrent and Distributed Systems

Synchronisation

Dr Daniele Scrimieri

Outline

- Synchronisation
- Modelling concurrency as interleaving
- The problem of interference
- Mutual exclusion

Synchronising Concurrent Processes

- To cooperate, the processes in a concurrent program must **communicate** with each other
- Communication can be programmed using shared variables or message passing:
 - when shared variables are used, one process writes into a shared variable that is read by another
 - when message passing is used, one process sends a message that is received by another
- *The main problem in concurrent programming is synchronising this communication*

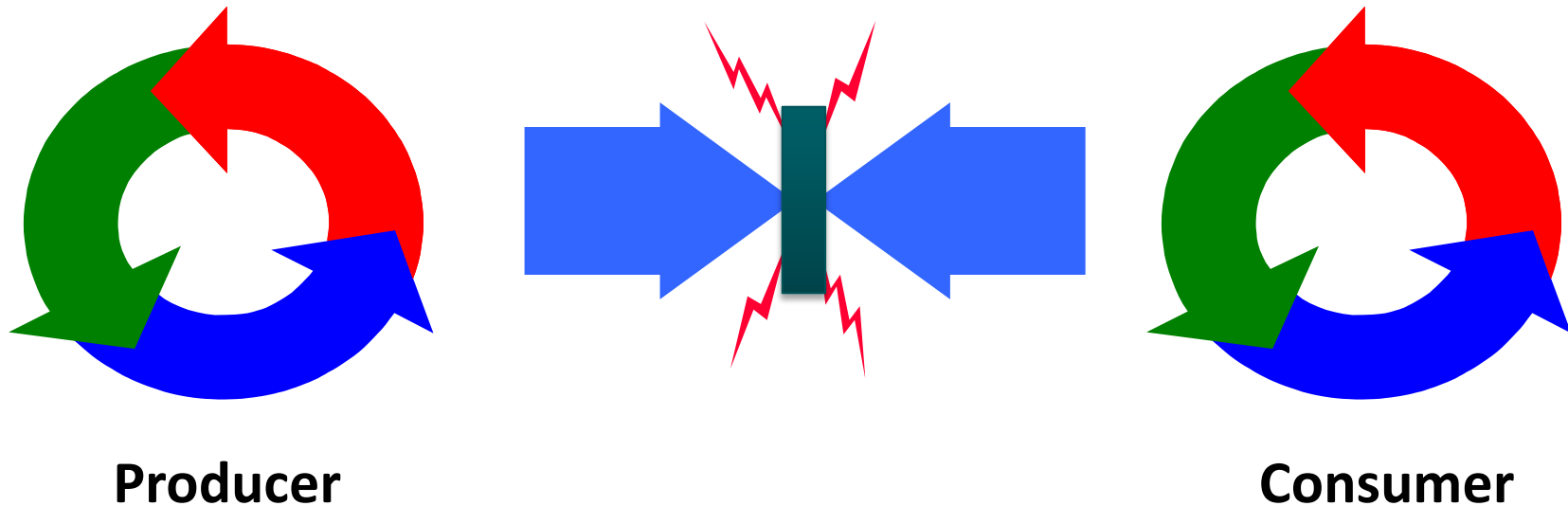
Synchronisation Problems

- There are two main synchronisation problems in concurrent programming:
 - **Mutual Exclusion**: ensuring that certain statements in different processes cannot execute at the same time
 - **Condition Synchronisation**: delaying a process until some (Boolean) condition is true. This is usually implemented by having one process wait for an event that is signalled by another process

Example: Shared Memory Location

Communication between a process that produces data and a process that uses it can be implemented using a *shared memory location*

- One process (the ***producer***) writes data to the memory location
- The other (the ***consumer***) reads data from the memory location



Shared Memory Synchronisation

Synchronisation conditions for the shared memory location:

- **Mutual exclusion** is necessary to ensure that the producer and consumer do not access the memory location at the same time
 - Partially written data must not be read
 - Partially read data must not be overwritten
- **Condition synchronisation** may be necessary to ensure that the consumer doesn't get too far ahead of the producer and vice versa
 - Data must not be read before it has been written or read twice
 - Data must not be overwritten before it has been read

Condition Synchronisation

- Of the two problems, **condition synchronisation** is the easier to solve
- The simplest solution is to use **busy waiting** – the process simply sits in a loop until the condition is true
 - For example, in the shared buffer problem, the consumer can loop repeatedly checking to see if there is a data item ready
 - There are other, more efficient, solutions which we will discuss in later lectures
 - Today we will focus on **mutual exclusion**
 - Java implementation for it (**synchronized** keyword)
 - **locks**

Sequential Programs

- All programs are sequential in that they execute a sequence of instructions in a pre-defined order
- There is a single thread of execution or control

$x = x + 1$



LOAD x

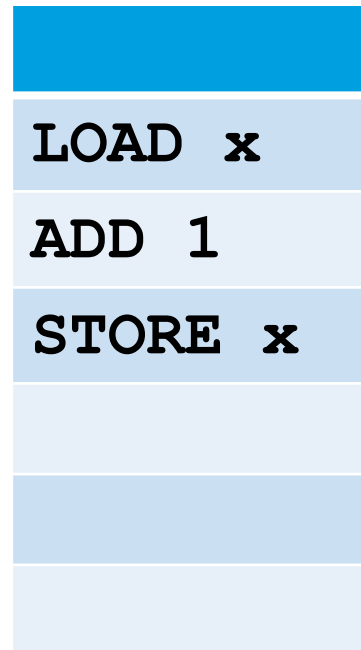
ADD 1

STORE x

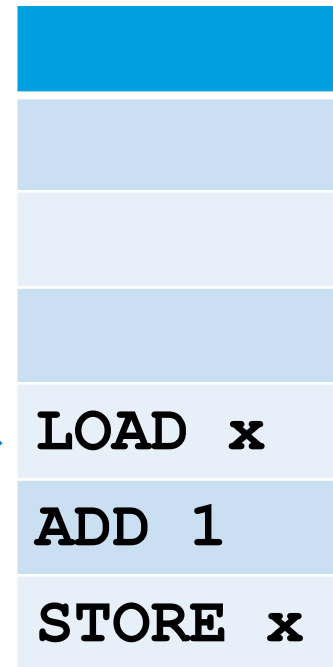
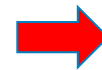
Concurrent Programs

- A **concurrent program** is one consisting of two or more processes - threads of execution or control
- Each one is itself a sequential program

$x = x + 1$



Process A



Process B

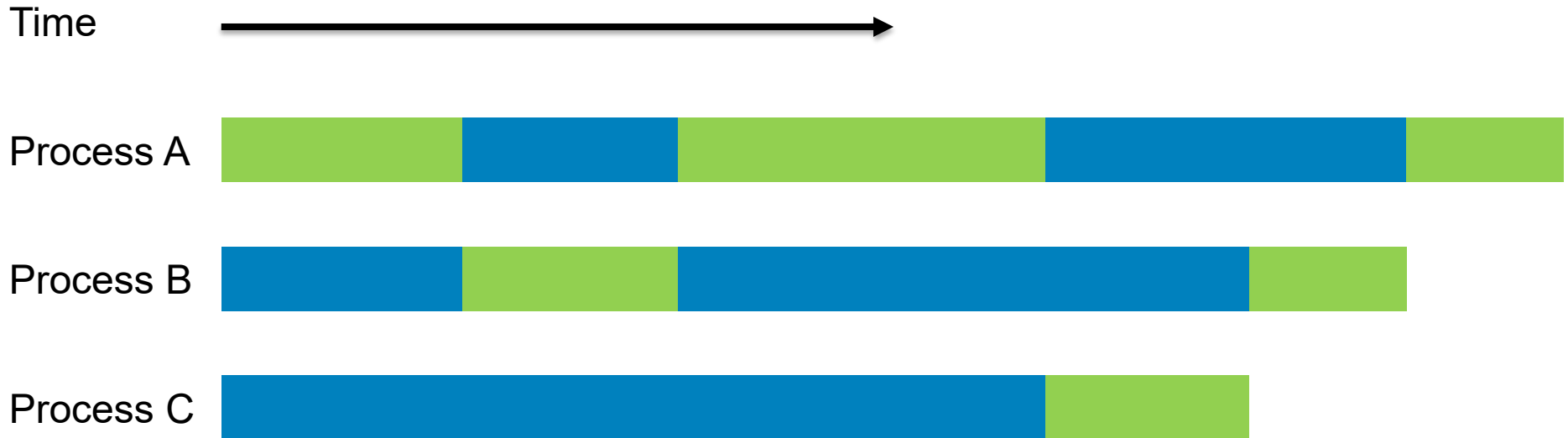
$x = x + 1$

Multiprogramming

- If we ignore pipelining, it is not possible for a single processor to execute more than one instruction at a time
- Thus, on the time scale of a single instruction, concurrent processes are not possible
- On a longer time scale, however, several processes may be interleaved so that each runs for a short time, then another is run, and so on
- Over a long enough time scale, the processes appear to run truly concurrently, although at any given point in time, only one of them is executing
- This is called **multiprogramming**

Interleaving Processes - Example

Consider a multiprogramming implementation of a concurrent program consisting of three processes:



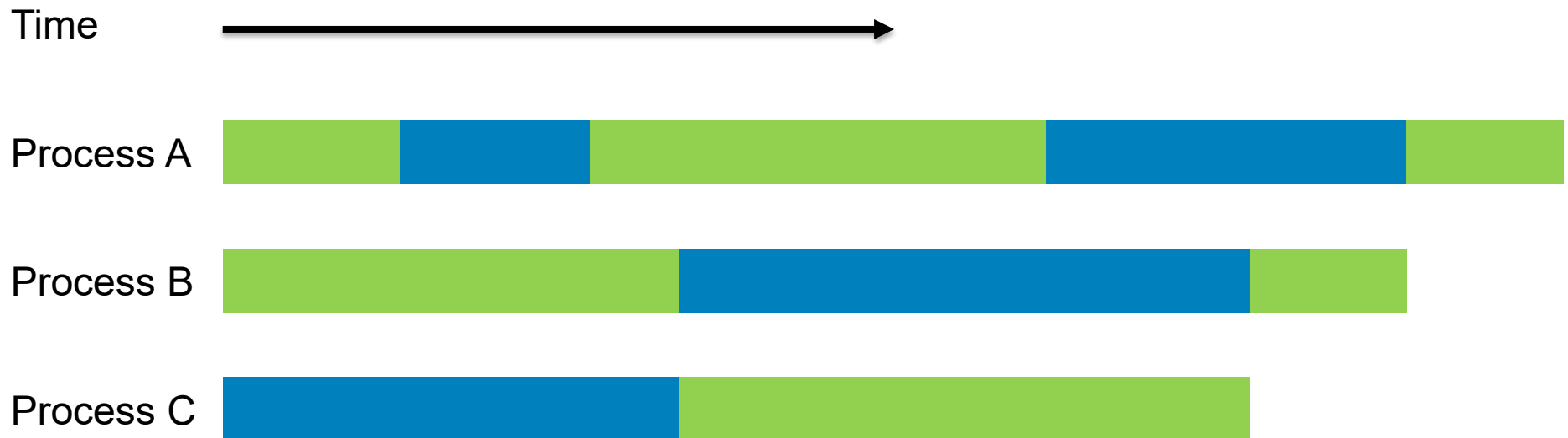
Interleaving (multiprogramming, one processor)

 Blocked  Running

• A, B, A, C, B, A

Interleaving and Overlapping Processes

Consider a multiprogramming implementation of a concurrent program consisting of three processes:



Interleaving and overlapping (multiprogramming, two processors)



- A, A, A
- B, C, B

Interleaving

- The switching between processes occurs
 - voluntarily (e.g., `yield()` in Java) or
 - in response to interrupts, which signal external events such as the completion of an I/O operation or clock tick to the processor
- The processor executes a sequence of instructions which is an interleaving of the instruction sequences from each process:
 - A, B, A, C, B, A
- Process switching does not affect the order in which instructions are executed by each process

Asynchronous Process Execution

- Processes usually have little or no control over how they are interleaved
- Application programmers can ignore the problems of timesharing the processes (that's done by the operating system)
- **Disadvantage**: processes effectively run **asynchronously**—we can't predict the relative speed with which they run, which runs first, at which point they will be suspended, etc.
 - This indeterminism makes debugging much more difficult than is the case for sequential programs

Multiprocessing Implementations

Multiprocessing implementations of concurrency can be modelled in the same way:

- Each program statement or machine instruction ultimately reduces to a sequence of **atomic actions** on the shared memory, e.g. loading and storing registers
- The effect of executing a set of atomic actions in parallel is equivalent to executing them in some arbitrary serial order
 - The state (memory) transformations caused by an atomic action are **indivisible**, hence cannot (by definition) be affected by atomic actions executed in parallel with it

Serialising Parallel Atomic Actions

- Two processes running on different processors can write to a shared memory location in parallel:
 - since writing is an atomic operation, one of the writes must go first
 - which actually goes first is determined by the Memory Management Unit (MMU)

Modelling Concurrency

- We therefore assume that:
 - concurrency is modelled as interleaving;
 - processes execute at arbitrary relative speeds—a process can take arbitrarily long to proceed from one instruction to the next;
 - instructions from processes are arbitrarily interleaved.
- This is referred to as an **asynchronous** model of execution.

Interference

- If instructions from different processes are **arbitrarily interleaved**, any interleaving which is not explicitly prohibited is allowed!
 - Inevitably, some interleavings will have undesirable results!
 - **Interference** occurs when two processes read and write shared variables in an unpredictable order, and hence with **unpredictable results**

Destructive update, caused by the arbitrary interleaving of read and write actions, is termed ***interference***.

Race Conditions

- A **race condition** occurs when
 - Multiple processes or threads read and write data items
 - They do so in a way where the final result depends on the order of execution of the processes
- The output depends on who finishes the race last

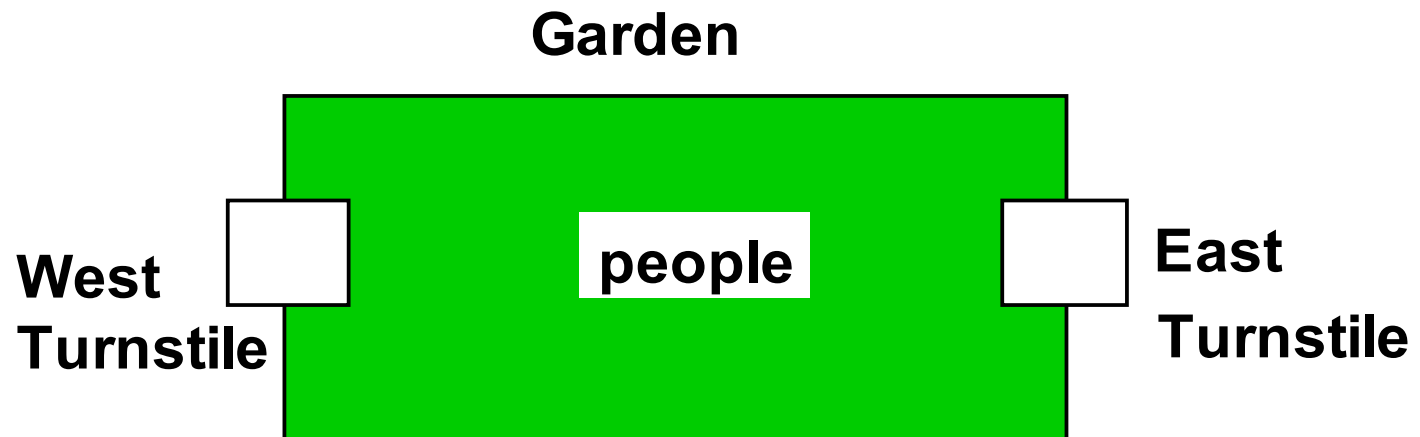
*“A **race condition** or **race hazard** is the condition of an electronics, software, or other system where the system's substantive behavior is dependent on the sequence or timing of other uncontrollable events. It becomes a bug when one or more of the possible behaviors is undesirable.”* (https://en.wikipedia.org/wiki/Race_condition)

Synchronisation Problems

- Booking systems
 - A central computer connected to remote terminals via communication links is used to automate seat reservations for a concert hall
 - To book a seat, a client chooses a free seat and the clerk enters the number of the chosen seat at the terminal and issues a ticket, if it is free
 - A system is required which avoids double bookings of the same seat whilst allowing clients free choice of the available seats
- Producer-consumer problems
 - Producer is producing some items, whereas there is one Consumer that is consuming the items produced by the Producer
 - The same memory buffer is shared by both producers and consumers which is of fixed-size

Ornamental Gardens Problem

- A large ornamental garden is open to members of the public who can enter through either of two turnstiles
- The owner of the garden writes a computer program to count how many people are in the garden at any one time
- The program has two processes, each of which monitors a turnstile and **increments a shared counter object** whenever someone enters via that processes' turnstile



Ornamental Gardens Program

```
// West turnstile
```

```
...
```

```
while(true) {  
    // wait for turnstile  
    count = count + 1;  
    // other stuff ...  
}
```

```
// East turnstile
```

```
...
```

```
while(true) {  
    // wait for turnstile  
    count = count + 1;  
    // other stuff ...  
}
```

```
count = 0;
```

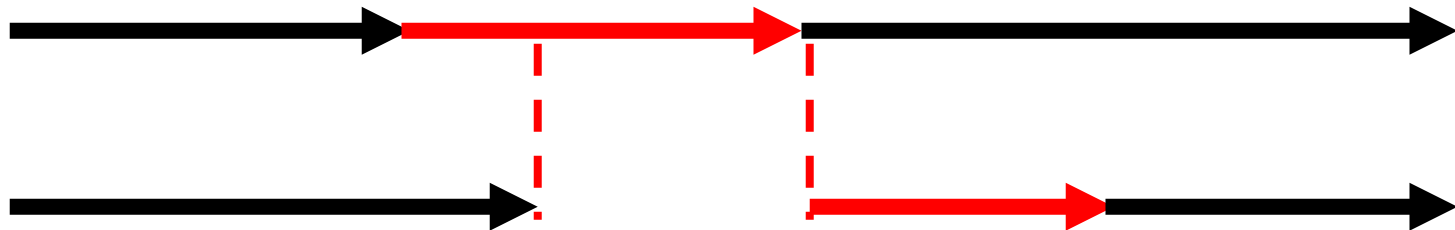
Example: Sequence of Actions

Action	Total	P1 (West) – r1	P2 (East) – r2
	0		
P1 loads		0	
P1 increments		1	
P2 loads			0
P2 increments			1
P2 stores	1		
P2 loads			1
P2 increments			2
P2 stores	2		
P1 stores	1		

3 persons enter by West, East, East gates. **Total: 1**

Avoiding Interference

- Interference bugs are extremely **difficult** to locate
- To avoid interference, we need to ensure that no two processes access a shared variable at the same time
- We do this by marking such sections of code as critical and requiring that no two processes execute **critical code** at the same time
- This is termed **mutual exclusion**
- Mutual exclusion (often referred to as “**mutex**”) can be modelled as atomic actions



Critical Sections

- A **critical section** is a section of code belonging to a process in a concurrent program that:
 - accesses a shared resource, e.g., a shared variable, shared communication channel, shared file etc.; and
 - for correct behaviour of the program only one process may access the shared resource at a time
- **Issues:**
 - How to select which process is allowed to access the critical section?
 - What happens to the other processes?

Locks and Waiting

Process 1:

<wait>

<critical section>

<signal>

Process 2:

<wait>

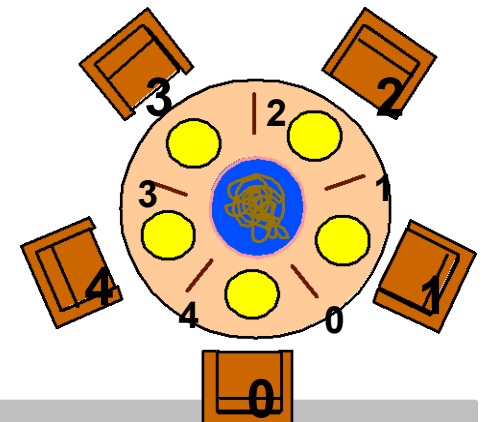
<critical section>

<signal>

Need atomic operation to implement wait!

Deadlock

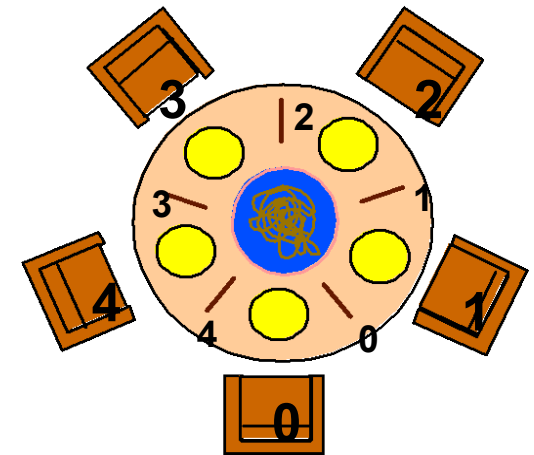
- Deadlock occurs when a process is waiting for an event that will never happen
- Necessary conditions for a deadlock to exist:
 - Processes claim exclusive access to resources
 - Processes hold some resources while waiting for others
 - Resources may not be removed from waiting processes
 - There exists a circular chain of processes in which each process holds a resource needed by the next process in the chain
- Example: **dining philosophers**



Competition among Processes for Resources

Main control problems:

- Interference
- Deadlock
- Starvation: when a process is repeatedly denied resources (like CPU time or memory) and can't make progress because higher-priority tasks always get them first



Enforcing Mutual Exclusion

Hardware solutions

- Disable interrupts
- Busy waiting

Software solutions

- Dekker's algorithm
- Peterson's algorithm
- Lamport's bakery algorithm
- Szymański's algorithm
- Taubenfeld's black-white bakery algorithm
- Maekawa's algorithm

Mutual exclusion devices

- Locks
 - Readers–writer locks
 - Recursive locks
- Semaphores
- Monitors
- Message passing
- Tuple space

Peterson's algorithm

```
bool flag[2] = {false, false};  
int turn;
```

```
P0:          flag[0] = true;  
P0_gate:     turn = 1;  
while (flag[1] == true && turn == 1)  
{  
    // busy wait  
}  
// critical section  
...  
// end of critical section  
flag[0] = false;
```

```
P1:          flag[1] = true;  
P1_gate:     turn = 0;  
while (flag[0] == true && turn == 0)  
{  
    // busy wait  
}  
// critical section  
...  
// end of critical section  
flag[1] = false;
```

Acknowledgments (and resources for further reading)

- Magee & Kramer (2006). Concurrency: State Models and Java Programs (2nd edition), John Wiley. Online slides <https://www.doc.ic.ac.uk/~jnm/book/>
- Brian Logan. Slides on Synchronisation